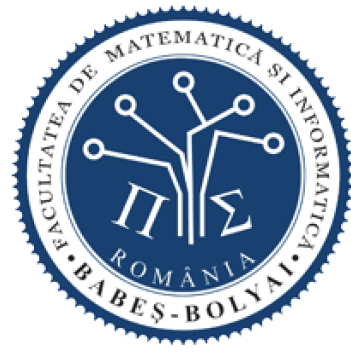




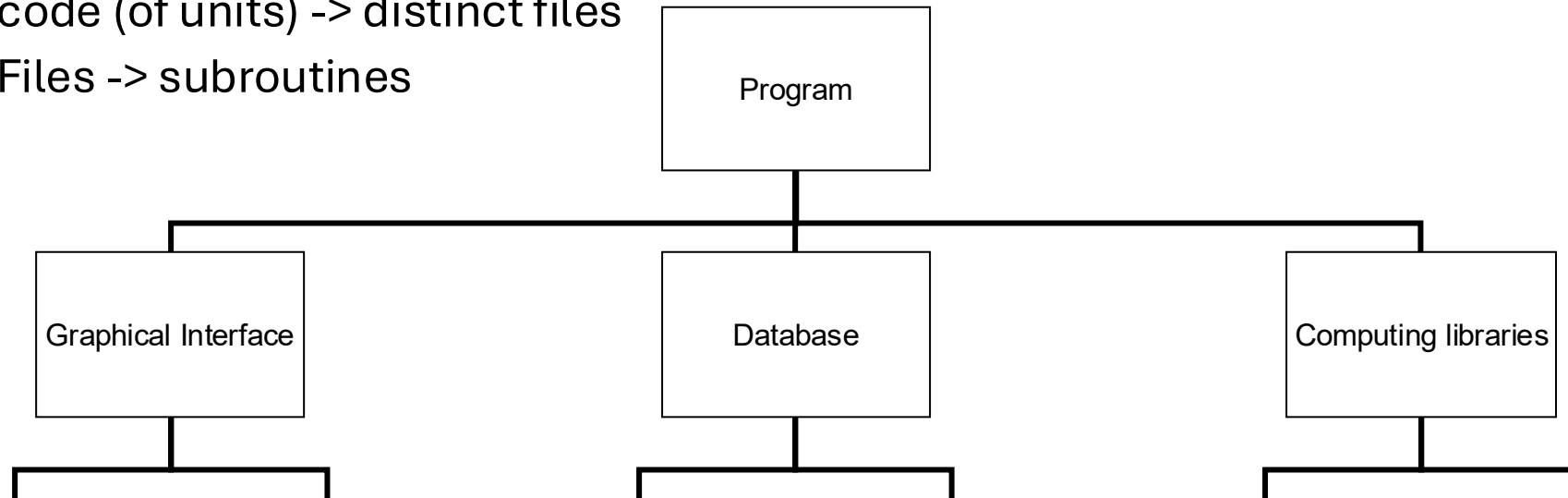
MLE5004: Computer systems architecture

VIII. Multimodule Programming

1. Modular Architectures

Modular programming

- How to split a problem in sub-problems?
 - Modularization
 - program -> logic units
 - code (of units) -> distinct files
 - Files -> subroutines



Modular programming

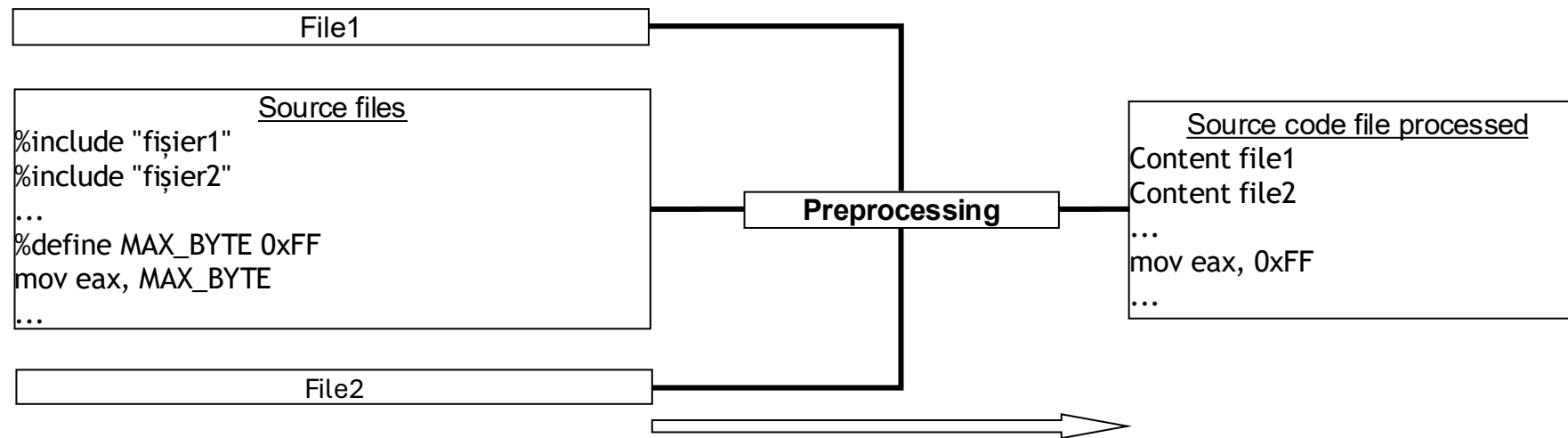
- Reusability
 - Code files
 - Code reusability and dates from assembly
 - %include directive
 - Binary files
 - Code reuse and ş dates from assembly
 - Codes and dates from high level languages
 - Libraries
 - Existence of separate binary files implies SEPARATE COMPILATION !!!

2. Techniques and tools

Techniques and tools

- Static inclusion at compilation/assembly: directive %include
 - Realize the specificity of language (but has equivalent also in other languages)
 - Modularization: allows only code division for the code written in the same language!
 - It is NOT multimodule programming !(this requires SEPARATE COMPILATION !!!)
 - Reusability: display the source code!
 - Dangerous and problematic:
 - Preprocessor mechanism -> textual concatenation of files
 - Exposes with visibility all names-> conflicts (redefinitions/redeclarations)
 - Include all the files – useful and unusefull!

Techniques and tools



Techniques and tools

- How to use %include

```
; files constante.inc
```

```
; double inclusion
```

```
%ifndef    _CONSTANTE_INC_ ; at first inclusion, _CONSTANTE_INC_ is not defined
```

```
%define    _CONSTANTE_INC_ ; definim _CONSTANTE_INC_ -> fake condition for next inclusions
```

```
; it is recommended that this type of files (included by other files) contain only  
declarations
```

```
MAX_BYTE    equ 0xFF
```

```
MAX_WORD    equ 0xFFFF
```

```
MAX_DWORD    equ 0xFFFFFFFF
```

```
MAX_QWORD    equ 0xFFFFFFFFFFFFFFFF
```

```
%endif ; _CONSTANTE_INC_
```


Techniques and tools

- %include usage example

```
; file constants.inc

; protect against double inclusion
#ifdef    _CONSTANTS_INC_ ; at the first inclusion, _CONSTANTE_INC_ is not defined
#define    _CONSTANTS_INC_ ; we define _CONSTANTE_INC_ -> the condition will be false for
future inclusions

; it is recommended that files included by other files should (only) consist of declarations!
MAX_BYTE    equ 0xFF
MAX_WORD    equ 0xFFFF
MAX_DWORD    equ 0xFFFFFFFF
MAX_QWORD    equ 0xFFFFFFFFFFFFFFFF

#endif ; _CONSTANTS_INC_
```

Techniques and tools

- %include usage example – “moving” the contents of eax in a BYTE/WORD/DWORD, according to the actual size of its value

```
; file program.asm
#include "constants.inc"
    cmp     eax, MAX_BYTE
    ja      .no_fit_in_byte           ; the value in eax fits in a BYTE?

.fits_in_byte:
    mov     [result_byte], al        ; if yes, save AL in result byte
    jmp     .done

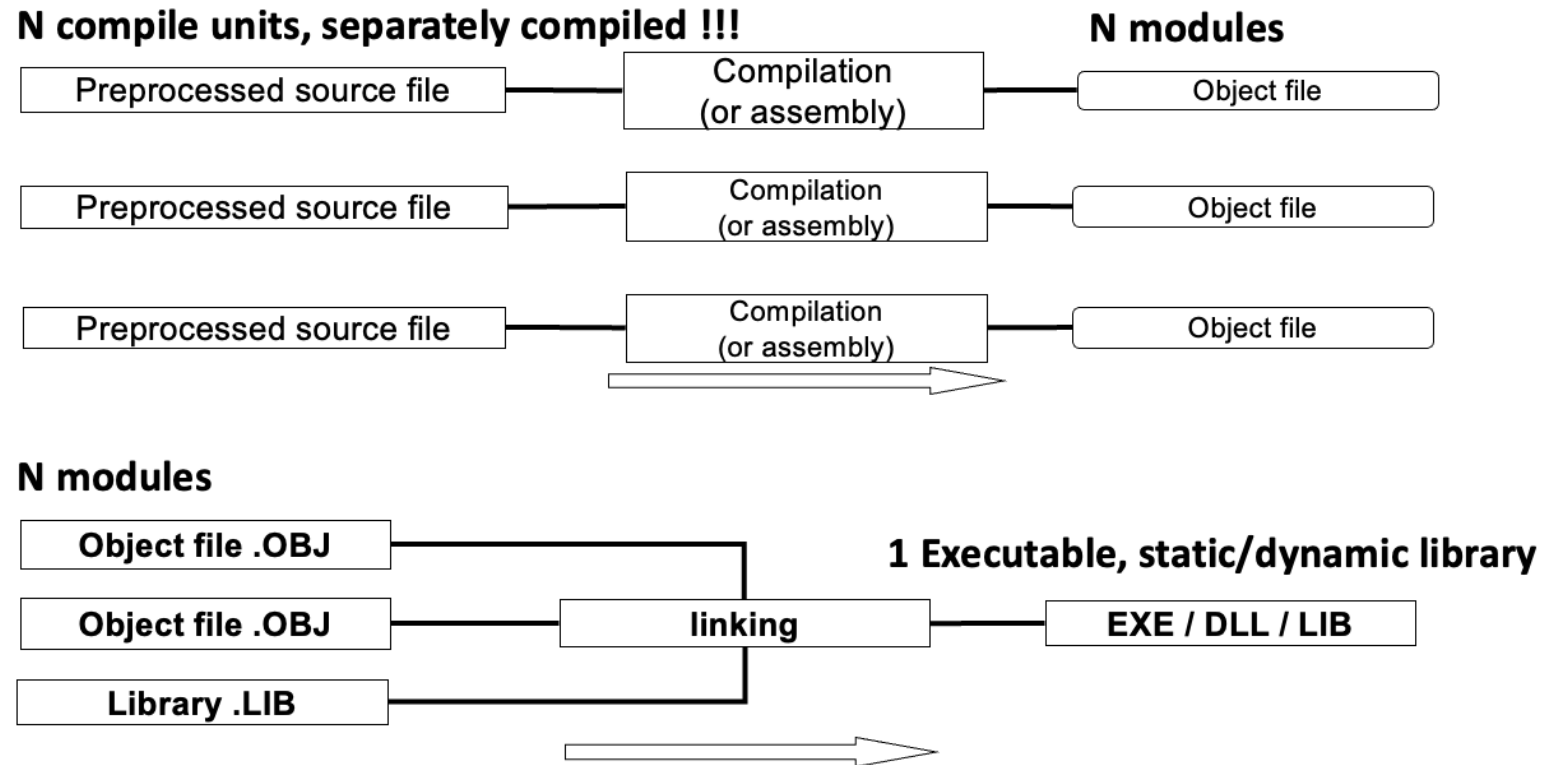
. no_fit_in_byte:
    cmp     eax, MAX_WORD
    ja      .no_fit_in_word         ; otherwise check if it fits in a WORD

.fits_in_word:
    mov     [resultat_word], ax      ; if yes, save AX in resultat_word
    jmp     .done

. no_fit_in_word:
    mov     [resultat_dword], eax    ; if a WORD is not enough, save all of eax
.done:
```

Techniques and tools

- Static linking at link-time
 - a step performed by the linker after assembly/compilation



Techniques and tools

- Static linking at link-time – summary of responsibilities
 - Preprocessor: text => text
 - Performs a processing of the source text, resulting a intermediary text source
 - Can be imagined as a component of the compiler or assembler
 - May be missing, many languages do not have a preprocessor.
 - Assembler: instructions (text) => binary encoding (object file)
 - Encodes the instructions and data (variables) from the preprocessed text source and builds an object file that consists of machine code and variable values, along with information about the content (variable names, subroutines, information about their type and visibility etc.)
 - Compiler: instructions (text) => binary encoding (object file)
 - Identifies sequences of processor instructions through which the functionalities described in the text source can be obtained, then, like an assembler, generates an object file that contains the binary codification of those instructions and the variables from the program
 - Assembling is a special case of compilation, where the processor instructions are provided directly in the text of the program and therefore the compiler does not need to select them!
 - Linking: object file => library or program
 - Constructs the final result, specifically a program (.exe) or a library (.dll or .lib) in which it links together (includes) the code and binary data from the object file
 - It has no regard for which compilers or languages were used! Linking only requires that the input file follow the standard format of object files!

Techniques and tools

- Static linking at link-time
 - Allows joining multiple binary modules (object files or static libraries) in a single file
 - Input: any number of object files (.OBJ) and/or static libraries (.LIB)
 - Attention: not all .LIB files are static libraries!
 - Output: .EXE or .LIB or .DLL (Dynamic-Link Library)
 - Multimodule: any number of files may be compiled separately and linked together
 - Step performed by the linker after compilation/assembly -> language independent
 - Reuse:
 - In binary form – does not expose the source code!
 - Allows inter-operability between different languages!
 - Other advantages and disadvantages:
 - The linker can identify and remove unused resources or perform other optimizations
 - Large program size: the program incorporates reused external resources
 - Large program size: popular libraries duplicated in most programs
 - NASM: global and extern directives
 - global name – allows the possibility of external reuse of this resource through its name
 - extern name – requests access to the specified resource; it needs to be public!

Techniques and tools

- Static linking at link-time – nasm requirements
 - Resources are shared based on a mutual agreement
 - Export through global name1, name2, ...
 - It makes the resources available to any interested file
 - Import through extern name1, name2, ...
 - Request access, no matter from which file the source is provided
 - Request without availability = error!
 - Only resources that are exported somewhere can be imported
 - Availability without request is allowed. Why?
 - Answer: even if none of the program's modules does not request/use a resource, it may be used in a future version or by a different program
- High level programming languages also offer syntactical constructions with an equivalent purpose!
 - Example: in C
 - Availability is automatic/implicit, however access may be restricted by using the keyword static
 - Access request is (also) done through the keyword extern

Techniques and tools

- Static linking at link-time – nasm requirements
 - Practical example for global and extern directives

; FILE1.ASM

global Var1, Subroutine2

extern Var3, Subroutine3

Subroutine1:

....

call(Subroutine3)

....

operations(Var3)

....

Subroutine2:

....

Var1 dd ...

Var2 db ...

; FILE2.ASM

extern Var1, Subroutine2

global Subroutine3, Var3

Subroutine3:

....

call(Subroutine2)

....

operations (Var1)

....

Subroutine1:

....

Var2 db ...

Var3 dd ...

Techniques and tools

- Static linking at link-time – nasm requirements
 - Practical example for global and extern directives

; FILE1.ASM

global Var1, Subroutine2

extern Var3, Subroutine3

Subroutine1:

....

call (Subroutine3)

....

operations(Var3)

....

Subroutine2:

....

Var1 dd ...

Var2 db ...

; FILE2.ASM

extern Var1, Subroutine2

global Subroutine3, Var3

Subroutine3:

....

call (Subroutine2)

....

operations(Var1)

....

Subroutine1:

....

Var2 db ...

Var3 dd ...

*Can reuse names
as long as they
are not global!*

Techniques and tools

- Example of a multimodule program nasm + nasm

```
; MODULE MAIN.ASM
global SirFinal
extern Concatenare

import printf msvcrt.dll
import exit msvcrt.dll
extern printf, exit
global start

segment code use32 public code class='code'
start:
    mov eax, Sir1
    mov ebx, Sir2
    call Concatenare
    push dword SirFinal
    call [printf]
    add esp, 1*4
    push dword 0
    call [exit]

segment data use32
Sir1 db 'Buna ', 0
Sir2 db 'dimineata!', 0
SirFinal resb 1000 ; space for result

; MODULE SUB.ASM
extern SirFinal
global Concatenare

segment code use32 public code class='code'
; eax = address of the first array,
; ebx = address of the second array
Concatenare:
    mov edi, SirFinal ; destination = SirFinal
    mov esi, eax ; source = first array
.sir1Loop:
    lodsb ; next byte
    test al, al ; array terminator (=0)?
    jz .sir2 ; if yes, go to second array
    stosb ; (otherwise) copy in destination
    jmp .sir1Loop ; continue to nul

.sir2:
    mov esi, ebx ; source = second array
.sir2Loop:
    lodsb ; same process for the new array
    test al, al
    jz .gata
    stosb
    jmp .sir2Loop

.gata:
    stosb ; add array terminator from al
    ret
```

Techniques and tools

- Example of a multimodule program nasm + nasm
 - Necessary steps to build the final executable program
 - Assemble file main.asm
 - `nasm.exe -fobj main.asm`
 - Assemble file sub.asm
 - `nasm.exe -fobj sub.asm`
 - Edit links between the two modules
 - `alink.exe main.obj sub.obj -oPE -entry:start -subsys:console`
 - Notice: the two modules can be assembled in any order! Only at linking is necessary that the referred symbols to have all implementations available in one of the object files offered by the linker
 - Linking, of course, is possible only after assembly/compilation!

Techniques and tools

- Static linkage at linking: nasm + high level languages
 - Requirements of the linker
 - global directive to allow access to other languages to our labels
 - extern directive to allow access in NASM of resources implemented in other languages
 - Declaration of variables and subroutines written in NASM in high level languages
 - Example C: extern declarator!
 - Entering the procedure
 - Keeping register values unaltered
 - Transmission and accessing parameters
 - Space allocation for local data (optional)
 - Returning a result (optional)
- The last aspects are discussed in detail in the call convention section!
 - See interface with high level languages: call conventions

Techniques and tools

- Example of a multimodule program asm + C

```
//
// AFISARE.C
//

// requests the C preprocessor to include file stdio.h
// stdio.h declares the header (return type and parameters) of the C function printf
#include <stdio.h>

// we declare the function from the asm file so that the C compiler knows the type of the parameters and the return var
// linkeditor will handle the function implementation, the compiler needs to know only the header
void asm_start(void); // equivalent with extern void asm_start(void) ! Any function declared at the most exterior level
// of a C module belongs to the extern memory class

// print function called by asm code
void afisare(int *vector, int numar_elemente) // any function defined at the most exterior level of a C module
{
    // is implicitly "global" - meaning it is
    automatically exported
    {
        int index;
        for (index = 0; index < numar_elemente; index++)
        {
            printf("%d", vector[index]);
        }
        printf("\n");
    }
}

// main program, calls asm_start function written in assembly
void main(void) // here starts the execution of the final program
{
    asm_start(); // we call the function from the asm file
}
```

Techniques and tools

- Example of a multimodule program asm + C

```
;
; VECTOR.NASM
;

; inform the assembler on the existence of the printing function
extern _afisare                ; add _ as prefix to names from C!

; inform the assembler that we want asm_start to be available to other compiling units
global _asm_start             ; add _ as prefix to names referred by C!

; asm code is available in a public segment, and can be shared with an other extern code
segment code public code use32

_asm_start:
    push dword elemente ; parameter by value (write in stack value 5)
    push dword vector   ; vector given by reference (write in stack its address)
    call _afisare        ; call C function, again with prefix _
    add esp, 4*2         ; afisare is a C function (cdecl) -> we need to free arguments!
    ret                 ; back to C code that called this code

; the linker can use the public data segment even for outside data
segment data public data use32
    vector dd 1, 2, 3, 4, 5 ; the vector we will print using the C routine
    elemente equ ($ - vector) / 4 ; constant equal to 5 (number of elements from the vector)
```

Techniques and tools

- Example of a multimodule program asm + C
 - Why _ ?
 - Build executable:
 - Compile/assembly :
 - afisare.c can be compiled with any C compiler (as desired) -> afisare.obj
 - Visual C: `cl /c afisare.c`
 - `nasm.exe vector.asm -fwin32 -o vector.obj`
 - Link-editing :
 - Call any linkeditor compatible with C, using:
 - Input: `afisare.obj` and `vector.obj`
 - Output: console application
 - `link vector.obj afisare.obj /OUT:afisare.exe /MACHINE:X86 /SUBSYSTEM:CONSOLE`
 - Alternatively, files can be bundled into one Visual Studio 'solution', configuring the IDE to:
 - Assemble the asm file: specifying for example as Pre-Build Event the above assembly command (`nasm.exe vector.asm -fwin32 -o vector.obj`)
 - Include `afisare.obj` as additional input to linkediting
 - There are Visual Studio extensions that solve this automatically and transparently!

3. Interfacing with high-level languages

Interfacing with high-level languages

- Passing parameters by value
 - Copying the values of the bytes that make up the data
 - Wasteful and slow when the data takes up a lot of memory!
 - From the caller's perspective, the data is constant!
- Passing parameters by reference (address/pointer)
 - Specifying the address (and sometimes the size) of the data
 - Inefficient when the data takes up little space
 - 32 extra bits to store the address + read the pointer before accessing the data
 - Data can be modified
- Decision to pass by value or by reference
 - Belongs to the called function, not decided/discussed at the call!
 - Performance criterion: size in bytes
 - Risk of exceeding available memory? By reference!
 - Accessibility criterion: must the data be modified?
 - Necessarily constant? By value!

Interfacing with high-level languages

- Call conventions
 - How do we pass parameters to subroutines?
 - What types of parameters can be passed?
 - In what order?
 - How many parameters? Any number?
 - What resources are volatile (can the called function alter them)?
 - Where is the result found?
 - What cleanup actions are required after the call?
 - Who is responsible for performing them?
- Conventions
 - Decided (and documented) by the called function, not by the caller!
 - Common: CDECL, STDCALL, FASTCALL
 - Rarely used or obsolete: PASCAL, FORTRAN, SYSCALL, etc...
 - User: in assembly all aspects documented by a convention are accessible to the programmer!
 - But we only have access from (other) assembly code!

Interfacing with high-level languages

- Calling conventions – C convention (**CDECL**)
 - Specific to the C language
 - How do we pass parameters to subroutines? By pushing them onto the stack
 - What types of parameters can be passed? Any, but require extended to a minimum of DWORD
 - In what order? Right to left, i.e., reverse the order of the declaration
 - How many parameters? Any? Yes, C allows functions with any number of parameters (e.g. printf)
 - What resources are volatile? EAX, ECX, EDX, Eflags (*pointers only)
 - Where is the result found? EAX, EDX:EAX or ST0 (FPU)
 - What cleanup actions are required? Argument release
 - Who is responsible for performing them? The caller!

Interfacing with high-level languages

- Calling conventions – STDCALL convention
 - Specific to the Windows operating system
 - Also called WINAPI
 - Used by Windows system libraries
 - Very similar to the CDECL convention
 - Differences:
 - Fixed number of parameters
 - Arguments are released by the called function

Interfacing with high-level languages

- Calling subroutines
 - Steps:
 1. Call code: preparing and executing the call
 2. Entry code: entering the procedure and preparing for execution
 3. Exit code: returning and releasing expired resources
- The actions depend on the calling convention of the called subroutine – but the steps remain the same!
- The steps are handled/implemented automatically in the code generated by the compilers of higher-level languages
 - In assembly, everything is left to us!

Interfacing with high-level languages

- Calling subroutines – call code
 - Tasks:
 1. Save volatile resources in use: register push
 2. Ensure constraints are respected (ESP aligned, DF=0, ...)
 3. Prepare arguments (stack/registers, according to convention): mov/push
 4. Make call: call
 - call subroutine when subroutine is statically linked
 - call [subroutine] if dynamic (at link-time)
 - call register or call [variable] for runtime dynamics
- Asm subroutines used only from asm can avoid (for simplicity and/or efficiency) these tasks

Interfacing with high-level languages

- Calling subroutines – calling code
 - Example: calling printf from asm to display numbers 0..9

```
import exit msvcrt.dll
import printf msvcrt.dll
extern exit, printf
global start
```

```
segment code use32
start:
    mov ecx, 10
    xor eax, eax
```

```
.next:
```

```
    push eax
    push ecx
```

```
    push eax
    push dword format_string
```

```
    call [printf]
```

```
    add esp, 2*4
```

```
    pop ecx
```

```
    pop eax
```

```
    inc eax
```

```
loop .next
```

```
push dword 0
```

```
call [exit]
```

```
segment data use32
format_string db "%d", 10, 13, 0
```

If the call was made from C, the compiler would have generated the calling code itself!
Since the call is made from assembly, the calling code must be written by us!

(1) Save volatile resources

(2) DF=0, stack was used only on DWORD

(3) Preparing CDECL call arguments

(4) Making call

Restore volatile resources (caller)

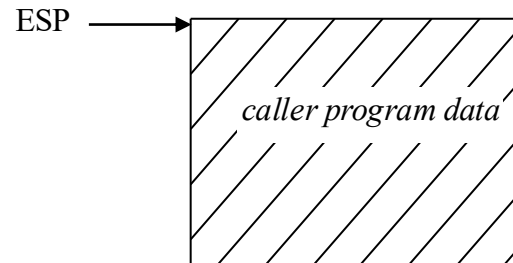
Interfacing with high-level languages

- Calling Subroutines – Call Code
 - The Effect of Call Code on the Stack

```
push eax  
push ecx
```

```
push eax  
push dword format_string  
call [printf]  
add esp, 2*4
```

Initial state



Interfacing with high-level languages

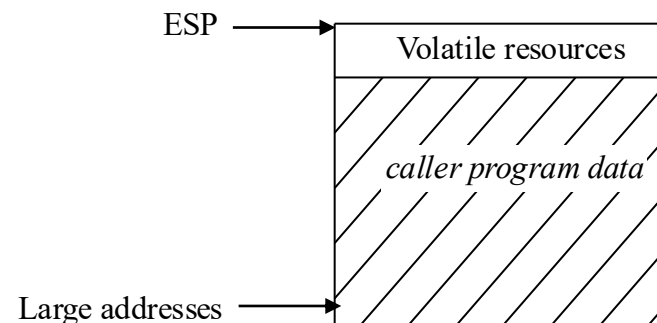
- Calling Subroutines – Call Code
 - The Effect of Call Code on the Stack

```
push eax  
push ecx
```

—————→ (1) Save volatile resources

```
push eax  
push dword format_string  
call [printf]  
add esp, 2*4
```

(1) Save volatile resources



Interfacing with high-level languages

- Calling Subroutines – Call Code
 - The Effect of Call Code on the Stack

```
push eax  
push ecx
```

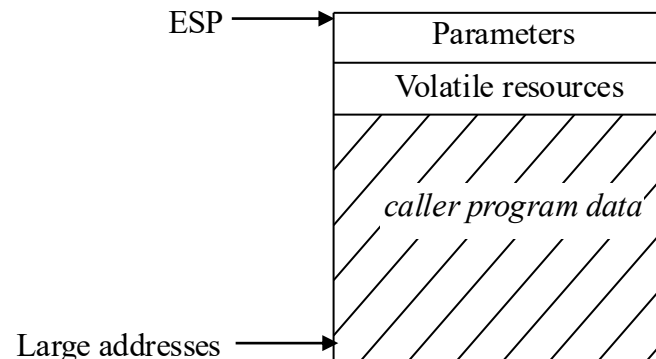
—————→ (1) Save volatile resources

```
push eax  
push dword format_string  
call [printf]  
add esp, 2*4
```

(2) DF=0, stack was used only on DWORD

—————→ (3) Prepare CDECL call arguments

(3) Prepare arguments for the call CDECL

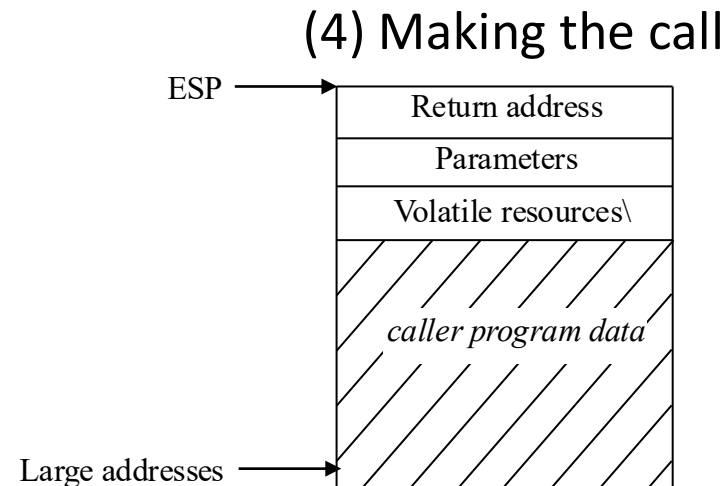


Interfacing with high-level languages

- Calling Subroutines – Call Code
 - The Effect of Call Code on the Stack

```
push eax  
push ecx  
  
push eax  
push dword format_string  
call [printf]  
add esp, 2*4
```

(1) Save volatile resources
(2) DF=0, stack was used only on DWORD
(3) Preparing CDECL call arguments
(4) Making call



Interfacing with high-level languages

- Calling subroutines – input code
 - Tasks:
 1. Setting up a stack frame: ebp or esp reference?
 2. Preparing local variables of the function: sub esp, number_of_bytes
 3. Saving a copy of modified non-volatile resources: push register
 - Any registers except volatile ones
- **Stack frame:** data structure stored on the stack, of fixed size (for a given subroutine) and containing:
 - Parameters prepared by the caller
 - Return address (to the instruction following the calling one)
 - Copies of non-volatile resources used by the subroutine
 - Local variables
- Asm subroutines used only from asm can avoid (for simplicity or efficiency) these tasks

Interfacing with high-level languages

- Calling subroutines – input code
 - Example: input code in the CDECL chdir function, generated by the C compiler

; disassembled with IDA - The Interactive Disassembler

; syntax corresponds to the MASM assembler

; int __cdecl chdir(const char *)

chdir:

mov edi, edi ; useless, but allows modification!

push ebp

mov ebp, esp

sub esp, 118h

mov eax, __security_cookie

xor eax, ebp

mov [ebp+var_4], eax

mov eax, [ebp+lpPathName]

and [ebp+var_110], 0

push ebx

or ebx, 0FFFFFFFFh

push esi

;

(1) Stack frame configuration

(2) Local variables

(3) Save non-volatile registers

The input code was automatically
generated by the compiler!

Interfacing with high-level languages

- Calling subroutines – entry code
 - Example: input code in the CDECL chdir function, generated by the C compiler

		<pre>factorial://in modulul C apelant se declara - extern int _stdcall factorial(int n)</pre>	
(1) Stack frame configuration	→	<pre>push ebp</pre>	
(2) Local variables	→	<pre>mov ebp, esp</pre>	; definim un cadru de stiva cu EBP pivot/referinta
	→	<pre>sub esp, 4</pre>	; alocam spatiu pe stiva pentru o variabila DWORD temporara
(3) Save non-volatile registers	→	<pre>push ebx</pre>	; salvam ebx pentru a-l putea restaura la sfarsit
		<pre>mov eax, [ebp + 8]</pre>	; incarcam valoarea argumentului din stackframe (n-ul curent)
		<pre>cmp eax, 2</pre>	; testam coditia de terminare (n < 2)
		<pre>jae .recursiv</pre>	
		<pre>mov eax, 1</pre>	; returnam 1 cand n < 2
		<pre>jmp .gata</pre>	
		<pre>.recursiv:</pre>	
		<pre>push eax</pre>	; retinem valoarea lui n sa nu-l pierdem la apelul recursiv
		<pre>dec eax</pre>	; pregatim noul parametru n-1 pt a apela factorial(n-1)
		<pre>push eax</pre>	; urcam in stiva valoarea n-1 ca parametru pt factorial(n-1)
		<pre>call factorial</pre>	; apel recursiv (STDCALL)
		<pre>mov [ebp - 4], eax</pre>	; salvam in DWORD-ul temporar valoarea lui factorial(n-1)
		<pre>pop eax</pre>	; restauram valoarea lui n
		<pre>mov ebx, [ebp - 4]</pre>	; ebx <- factorial(n-1), preluat din variabila temporara
		<pre>mul ebx</pre>	; calculam (edx:)eax = (EAX=)n * (EBX=)factorial(n-1)
		<pre>.gata:</pre>	
		<pre>pop ebx</pre>	; refacem EBX la valoarea initiala
		<pre>add esp, 4</pre>	; eliberam memoria ocupata de variabila temporara
		<pre>pop ebp</pre>	; restauram ebp la valoarea initiala
		<pre>ret 4</pre>	; STDCALL: revenire cu eliberare memorie parametri

Interfacing with high-level languages

- Calling subroutines – entry code
 - Example: input code in a STDCALL function written in asm –stackframe

```
factorial:
    push ebp                ← ; definim un cadru de stiva cu EBP pivot/referinta
    mov ebp, esp            ; alocam spatiu pe stiva pentru o variabila DWORD temporara
    sub esp, 4              ; salvam ebx pentru a-l putea restaura
    push ebx

    mov eax, [ebp + 8]      ; incarcam valoarea argumentului din stackframe

    cmp eax, 2              ; testam coditia de terminare (n < 2)
    jae .recursiv
    mov eax, 1              ; returnam 1 cand n < 2
    jmp .gata

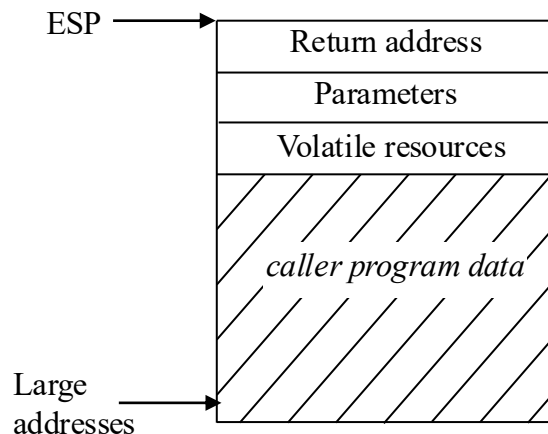
.recursiv:
    push eax                ; retinem valoarea lui n sa nu-l pierdem la apelul recursiv

    dec eax
    push eax                ; apelam factorial(n-1), am urcat in stiva valoarea n-1
    call factorial          ; apel recursiv (STDCALL)

    mov [ebp - 4], eax       ; salvam in DWORD-ul temporar valoarea lui factorial(n-1)
    pop eax                 ; restauram valoarea lui n
    mov ebx, [ebp - 4]       ; ebx <- factorial(n-1), preluat din variabila temporara
    mul ebx                 ; calculam (edx:)eax = (EAX=)n * (EBX=)factorial(n-1)

.gata:
    pop ebx                 ; refacem EBX la valoarea initiala
    add esp, 4              ; eliberam memoria ocupata de variabila temporara
    pop ebp                 ; restauram ebp la valoarea initiala
    ret 4                   ; STDCALL: revenire cu eliberare memorie parametri
```

State on entry



Interfacing with high-level languages

- Calling subroutines – entry code
 - Example: input code in a STDCALL function written in asm –stackframe

(1) Configure stack frame

```
factorial:
    push ebp                ; definim un cadru de stiva cu EBP pivot/referinta
    mov ebp, esp            ; alocam spatiu pe stiva pentru o variabila DWORD temporara
    sub esp, 4              ; salvam ebx pentru a-l putea restaura
    push ebx

    mov eax, [ebp + 8]      ; incarcam valoarea argumentului din stackframe

    cmp eax, 2              ; testam coditia de terminare (n < 2)
    jae .recursiv
    mov eax, 1              ; returnam 1 cand n < 2
    jmp .gata
```

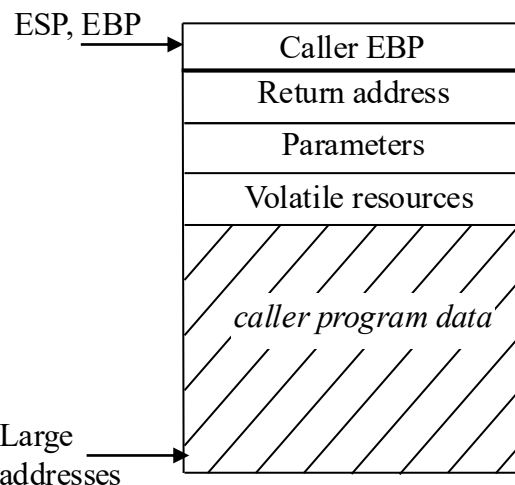
(1) Configure stack frame

```
.recursiv:
    push eax                ; retinem valoarea lui n sa nu-l pierdem la apelul recursiv

    dec eax
    push eax                ; apelam factorial(n-1), am urcat in stiva valoarea n-1
    call factorial          ; apel recursiv (STDCALL)

    mov [ebp - 4], eax       ; salvam in DWORD-ul temporar valoarea lui factorial(n-1)
    pop eax                 ; restauram valoarea lui n
    mov ebx, [ebp - 4]       ; ebx <- factorial(n-1), preluat din variabila temporara
    mul ebx                 ; calculam (edx:)eax = (EAX=)n * (EBX=)factorial(n-1)

.gata:
    pop ebx                 ; refacem EBX la valoarea initiala
    add esp, 4              ; eliberam memoria ocupata de variabila temporara
    pop ebp                 ; restauram ebp la valoarea initiala
    ret 4                   ; STDCALL: revenire cu eliberare memorie parametri
```



Interfacing with high-level languages

- Calling subroutines – entry code
 - Example: input code in a STDCALL function written in asm –stackframe

(1) Configure stack frame

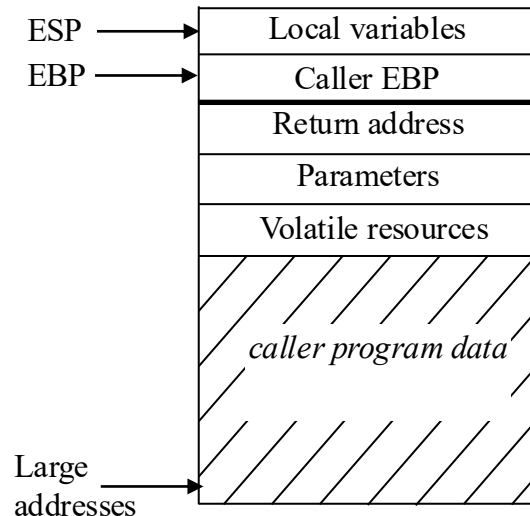
(2) Local variables

```
factorial:
    push ebp
    mov ebp, esp
    sub esp, 4
    push ebx

    mov eax, [ebp + 8]
    cmp eax, 2
    jae .recursiv
    mov eax, 1
    jmp .gata
```

; definim un cadru de stiva cu EBP pivot/referinta
; alocam spatiu pe stiva pentru o variabila DWORD temporara
; salvam ebx pentru a-l putea restaura
; incarcam valoarea argumentului din stackframe
; testam coditia de terminare (n < 2)
; returnam 1 cand n < 2

(2) Local variables



```
.recursiv:
    push eax
    dec eax
    push eax
    call factorial
    mov [ebp - 4], eax
    pop eax
    mov ebx, [ebp - 4]
    mul ebx

.gata:
    pop ebx
    add esp, 4
    pop ebp
    ret 4
```

; retinem valoarea lui n sa nu-l pierdem la apelul recursiv
; apeland factorial(n-1), am urcat in stiva valoarea n-1
; apel recursiv (STDCALL)
; salvam in DWORD-ul temporar valoarea lui factorial(n-1)
; restauram valoarea lui n
; ebx <- factorial(n-1), preluat din variabila temporara
; calculam (edx:)eax = (EAX=)n * (EBX=)factorial(n-1)
; refacem EBX la valoarea initiala
; eliberam memoria ocupata de variabila temporara
; restauram ebp la valoarea initiala
; STDCALL: revenire cu eliberare memorie parametri

- Calling subroutines – entry code
 - Example: input code in a STDCALL function written in asm –stackframe

- (1) Stack frame configuration
- (2) Local variables
- (3) Save non-volatile registers

```

push ebp
mov ebp, esp
sub esp, 4
push ebx

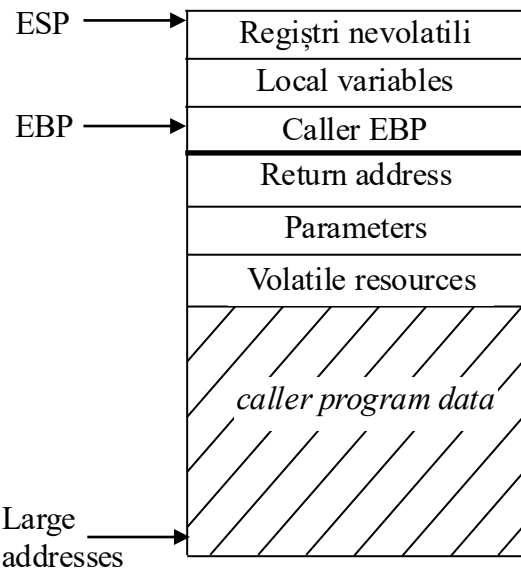
```

```
; definim un cadru de stiva cu EBP pivot/referinta
; alocam spatiu pe stiva pentru o variabila DWORD temporara
; salvam ebx pentru a-l putea restaura
```

```
mov eax, [ebp + 8] ; incarcam valoarea argumentului din stackframe
```

```
cmp eax, 2          ; testam coditia de terminare (n < 2)
jae .recursiv
mov eax, 1          ; returnam 1 cand n < 2
jmp .gata
```

(3) Save non-volatile registers



push eax

```
; retinem valoarea lui n sa nu-l pierdem la apelul recursiv
```

push eax

push eax

```
; apelam factorial(n-1), am urcat in stiva valoarea n-1
```

```
call factorial
```

```
; apel recursiv (STDCALL)
```

```
mov [ebp - 4], eax
```

```
; salvam in DWORD-ul temporar valoarea lui factorial(n-1)
```

```
pop    eax
```

```
; restauram valoarea lui n
```

```
mov ebx, [ebp - 4]
```

```
: ebx <- factorial(n-1), preluat din variabila temporara
```

```
mul    ebx
```

```

: calculam (edx:)eax = (EAX=)n * (EBX=)factorial(n-1)

```

.gata:

```
pop ebx
```

```
; refacem EBX la valoarea initiala
```

```
add esp, 4
```

```
; eliberam memoria ocupata de variabila temporara
```

```
pop ebp
```

```
    : restauram ebp la valoarea initiala
```

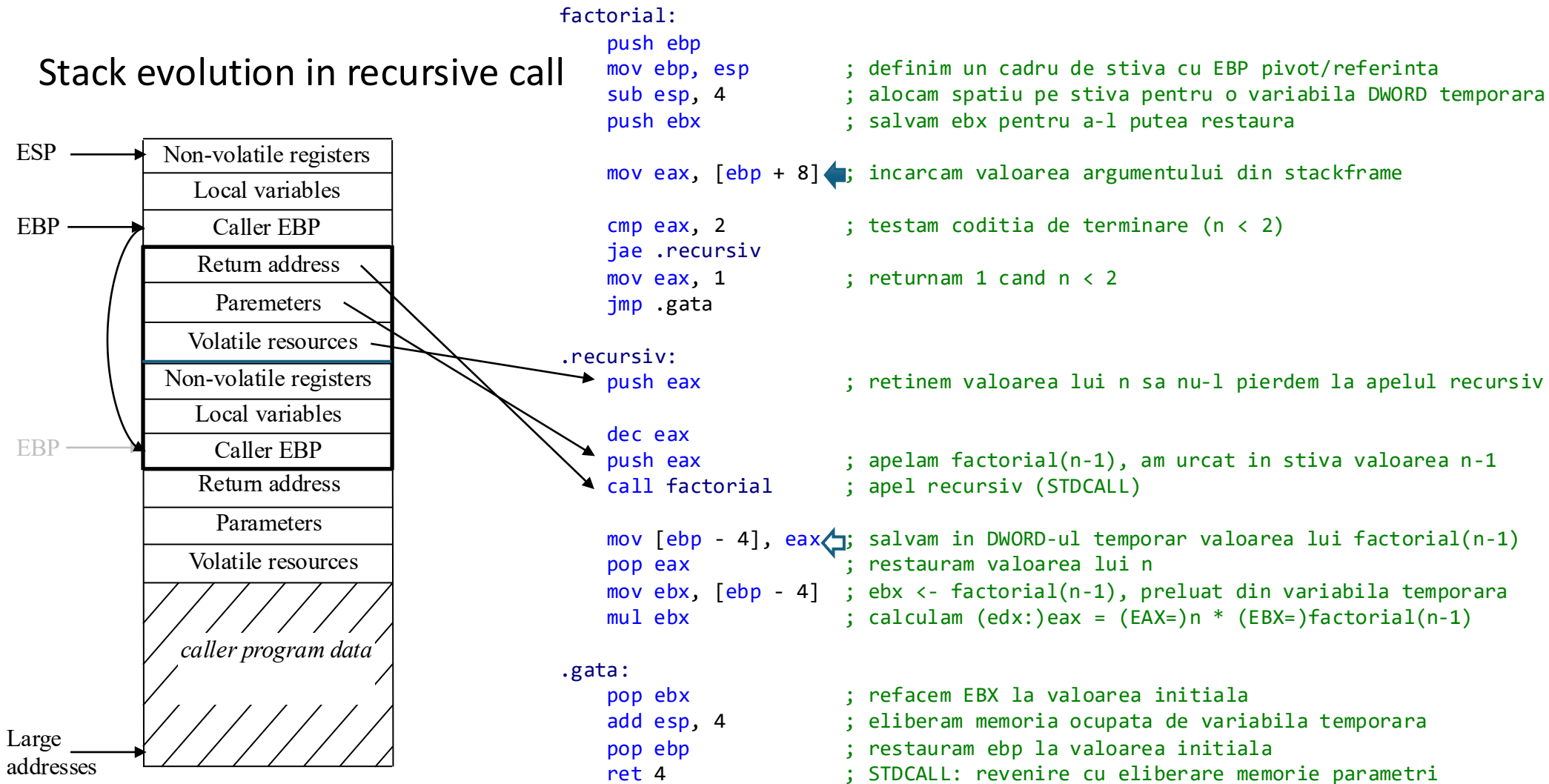
```
ret 4
```

```
: STDCALL: revenire cu eliberare memorie parametri
```

Interfacing with high-level languages

- Calling subroutines – entry code
 - Example: input code in a STDCALL function written in asm –stackframe

Stack evolution in recursive call

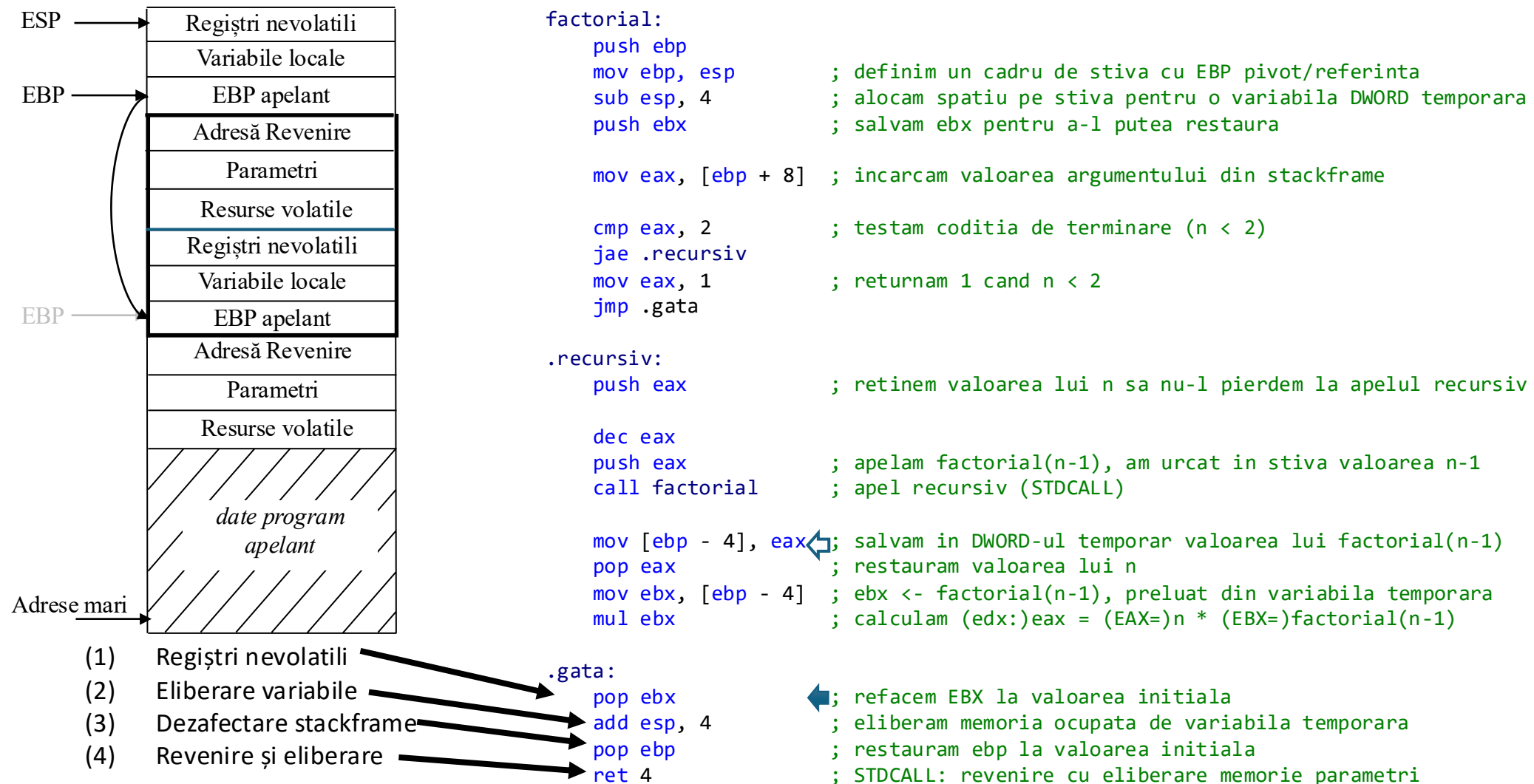


Interfacing with high-level languages

- Subroutine call – exit code
 - Tasks:
 1. Restore altered nonvolatile resources
 2. Release local function variables
 3. Deal with stack frame
 4. Return from function and release arguments
 - CDECL:
 - Called subroutine: ret
 - Calling subroutine: add esp, argument_size
 - STDCALL:
 - ret argument_size
- Except for volatile resources and direct function results, the state of the program after these steps must reflect the initial state, before the call!

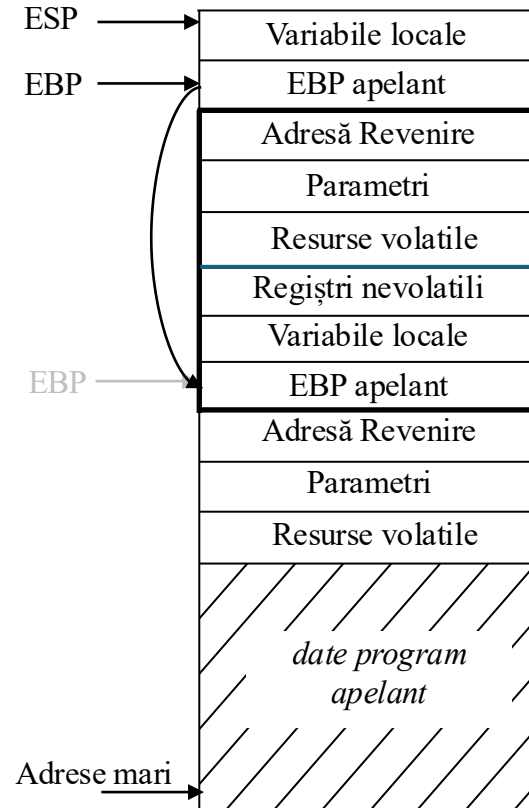
Interfacing with high-level languages

- Subroutine call – exit code
 - Example: exit code from a STDCALL asm function (from recursive call) -stackframe



Interfacing with high-level languages

- Subroutine call – exit code
 - Example: exit code from a STDCALL asm function (from recursive call) -stackframe



```
factorial:
    push ebp
    mov ebp, esp           ; definim un cadru de stiva cu EBP pivot/referinta
    sub esp, 4             ; alocam spatiu pe stiva pentru o variabila DWORD temporara
    push ebx               ; salvam ebx pentru a-l putea restaura

    mov eax, [ebp + 8]     ; incarcam valoarea argumentului din stackframe

    cmp eax, 2             ; testam coditia de terminare (n < 2)
    jae .recursiv          ; returnam 1 cand n < 2
    mov eax, 1
    jmp .gata

.recursiv:
    push eax               ; retinem valoarea lui n sa nu-l pierdem la apelul recursiv

    dec eax
    push eax               ; apelam factorial(n-1), am urcat in stiva valoarea n-1
    call factorial          ; apel recursiv (STDCALL)

    mov [ebp - 4], eax      ; salvam in DWORD-ul temporar valoarea lui factorial(n-1)
    pop eax                ; restauram valoarea lui n
    mov ebx, [ebp - 4]      ; ebx <- factorial(n-1), preluat din variabila temporara
    mul ebx                ; calculam (edx:)eax = (EAX=)n * (EBX=)factorial(n-1)

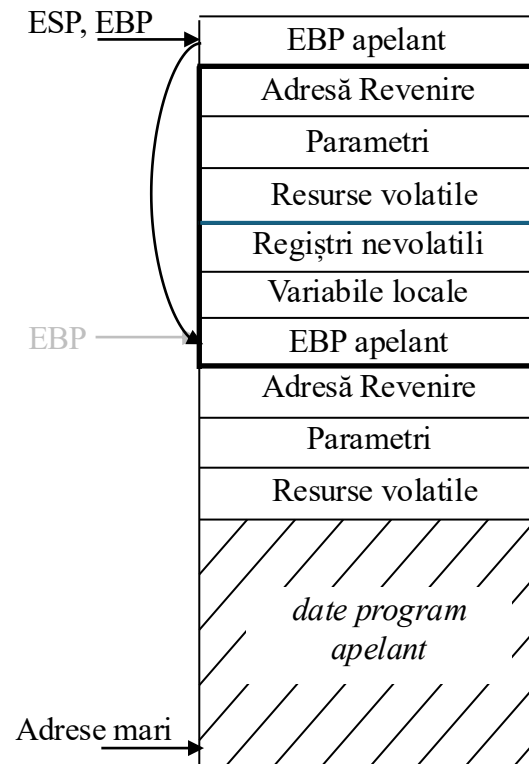
.gata:
    pop ebx                ; refacem EBX la valoarea initiala
    add esp, 4             ; eliberam memoria ocupata de variabila temporara
    pop ebp                ; restauram ebp la valoarea initiala
    ret 4                  ; STDCALL: revenire cu eliberare memorie parametri
```

(1) Regiștri nevolatili



Interfacing with high-level languages

- Subroutine call – exit code
 - Example: exit code from a STDCALL asm function (from recursive call) -stackframe



```
factorial:
    push ebp
    mov ebp, esp           ; definim un cadru de stiva cu EBP pivot/referinta
    sub esp, 4             ; alocam spatiu pe stiva pentru o variabila DWORD temporara
    push ebx               ; salvam ebx pentru a-l putea restaura

    mov eax, [ebp + 8]     ; incarcam valoarea argumentului din stackframe

    cmp eax, 2             ; testam coditia de terminare (n < 2)
    jae .recursiv
    mov eax, 1             ; returnam 1 cand n < 2
    jmp .gata
```

```
.recursiv:
    push eax               ; retinem valoarea lui n sa nu-l pierdem la apelul recursiv

    dec eax
    push eax               ; apelam factorial(n-1), am urcat in stiva valoarea n-1
    call factorial         ; apel recursiv (STDCALL)

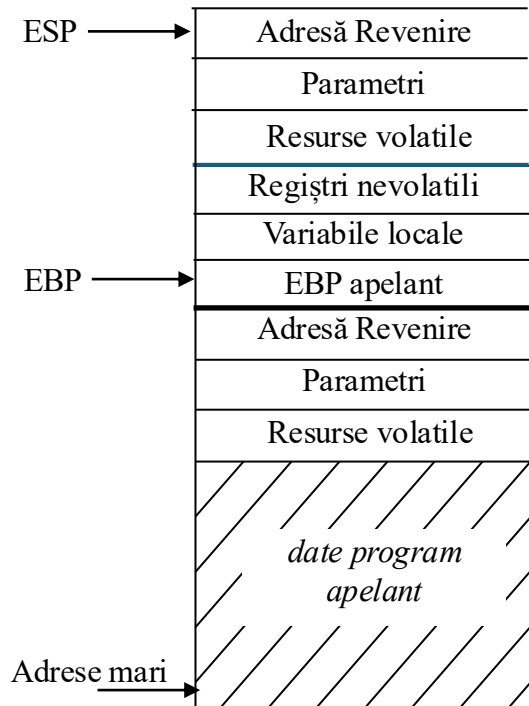
    mov [ebp - 4], eax     ; salvam in DWORD-ul temporar valoarea lui factorial(n-1)
    pop eax                ; restauram valoarea lui n
    mov ebx, [ebp - 4]     ; ebx <- factorial(n-1), preluat din variabila temporara
    mul ebx                ; calculam (edx:)eax = (EAX=)n * (EBX=)factorial(n-1)
```

(2) Variabile locale

```
.gata:
    pop ebx                ; refacem EBX la valoarea initiala
    add esp, 4             ; eliberam memoria ocupata de variabila temporara
    pop ebp                ; restauram ebp la valoarea initiala
    ret 4                  ; STDCALL: revenire cu eliberare memorie parametri
```

Interfacing with high-level languages

- Subroutine call – exit code
 - Example: exit code from a STDCALL asm function (from recursive call) -stackframe



```

factorial:
    push ebp
    mov ebp, esp           ; definim un cadru de stiva cu EBP pivot/referinta
    sub esp, 4             ; alocam spatiu pe stiva pentru o variabila DWORD temporara
    push ebx               ; salvam ebx pentru a-l putea restaura

    mov eax, [ebp + 8]     ; incarcam valoarea argumentului din stackframe

    cmp eax, 2             ; testam coditia de terminare (n < 2)
    jae .recursiv
    mov eax, 1             ; returnam 1 cand n < 2
    jmp .gata

.recursiv:
    push eax               ; retinem valoarea lui n sa nu-l pierdem la apelul recursiv

    dec eax
    push eax               ; apelam factorial(n-1), am urcat in stiva valoarea n-1
    call factorial          ; apel recursiv (STDCALL)

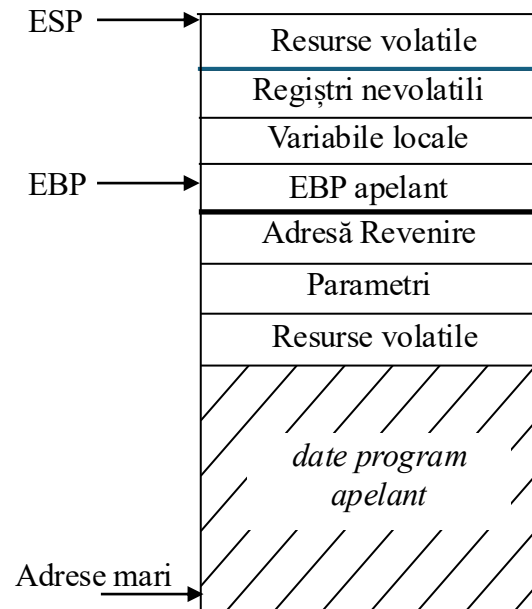
    mov [ebp - 4], eax      ; salvam in DWORD-ul temporar valoarea lui factorial(n-1)
    pop eax                ; restauram valoarea lui n
    mov ebx, [ebp - 4]      ; ebx <- factorial(n-1), preluat din variabila temporara
    mul ebx                ; calculam (edx:)eax = (EAX=)n * (EBX=)factorial(n-1)

.gata:
    pop ebx                ; refacem EBX la valoarea initiala
    add esp, 4             ; eliberam memoria ocupata de variabila temporara
    pop ebp                ; restauram ebp la valoarea initiala
    ret 4                  ; STDCALL: revenire cu eliberare memorie parametri
    
```

(3) Stackframe

Interfacing with high-level languages

- Subroutine call – exit code
 - Example: exit code from a STDCALL asm function (from recursive call) -stackframe



```

factorial:
    push ebp
    mov ebp, esp        ; definim un cadru de stiva cu EBP pivot/referinta
    sub esp, 4          ; alocam spatiu pe stiva pentru o variabila DWORD temporara
    push ebx            ; salvam ebx pentru a-l putea restaura

    mov eax, [ebp + 8]   ; incarcam valoarea argumentului din stackframe

    cmp eax, 2          ; testam coditia de terminare (n < 2)
    jae .recursiv
    mov eax, 1          ; returnam 1 cand n < 2
    jmp .gata

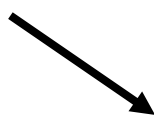
.recursiv:
    push eax            ; retinem valoarea lui n sa nu-l pierdem la apelul recursiv

    dec eax
    push eax            ; apelam factorial(n-1), am urcat in stiva valoarea n-1
    call factorial      ; apel recursiv (STDCALL)

    mov [ebp - 4], eax   ; salvam in DWORD-ul temporar valoarea lui factorial(n-1)
    pop eax             ; restauram valoarea lui n
    mov ebx, [ebp - 4]   ; ebx <- factorial(n-1), preluat din variabila temporara
    mul ebx             ; calculam (edx:)eax = (EAX=)n * (EBX=)factorial(n-1)

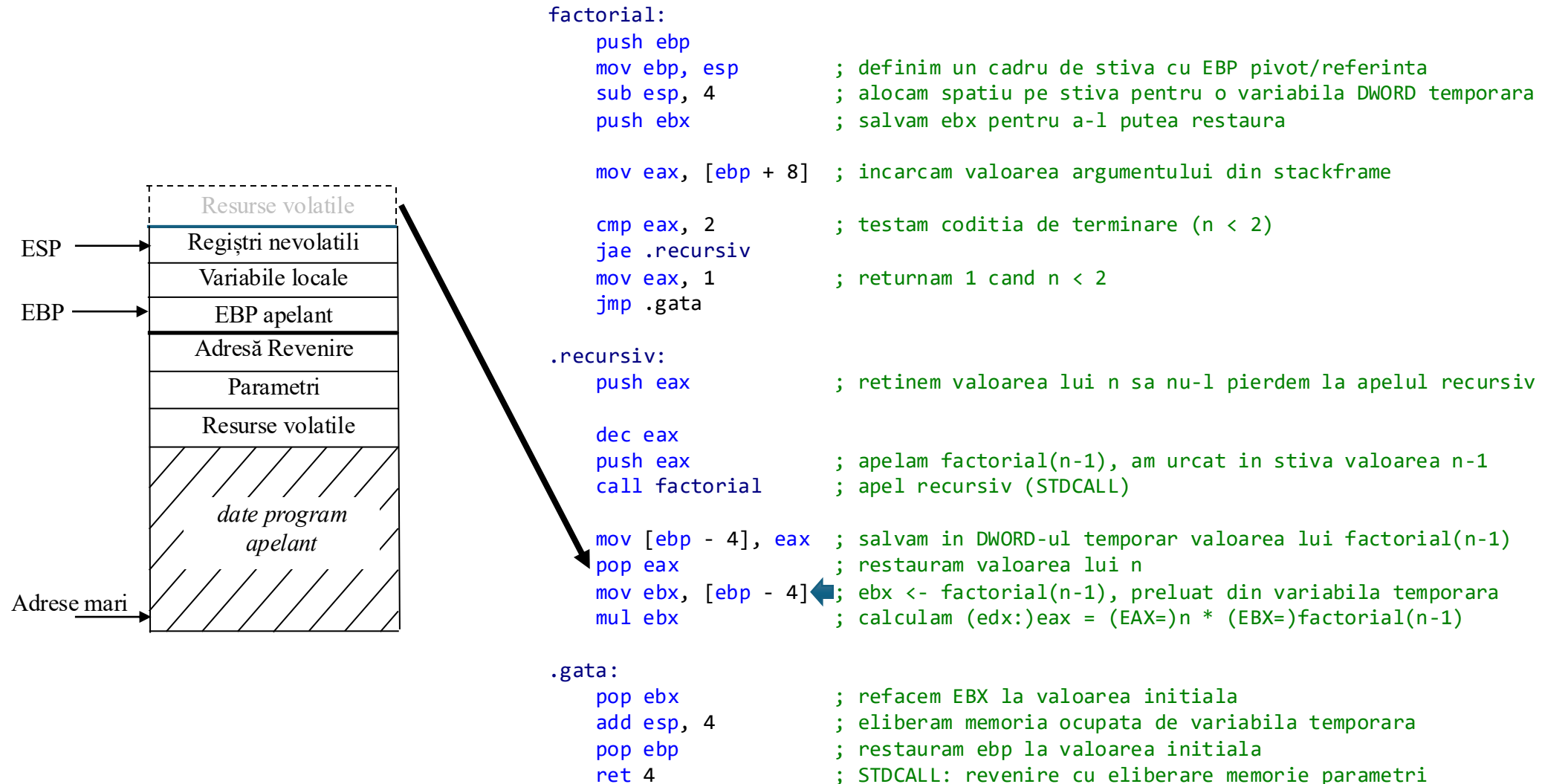
.gata:
    pop ebx            ; refacem EBX la valoarea initiala
    add esp, 4          ; eliberam memoria ocupata de variabila temporara
    pop ebp            ; restauram ebp la valoarea initiala
    ret 4              ; STDCALL: revenire cu eliberare memorie parametri
    
```

(4) Revenire și eliberare



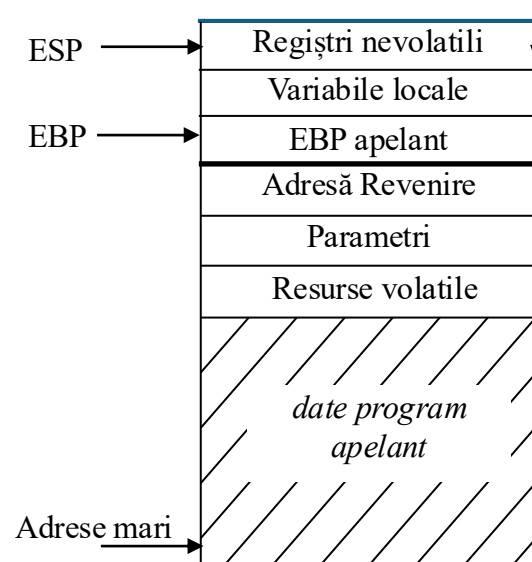
Interfacing with high-level languages

- Subroutine call – exit code
 - Example: exit code from a STDCALL asm function (from recursive call) -stackframe



Interfacing with high-level languages

- Subroutine call – exit code
 - Example: exit code from a STDCALL asm function (from recursive call) -stackframe



```
factorial:
    push ebp
    mov ebp, esp           ; definim un cadru de stiva cu EBP pivot/referinta
    sub esp, 4             ; alocam spatiu pe stiva pentru o variabila DWORD temporara
    push ebx               ; salvam ebx pentru a-l putea restaura

    mov eax, [ebp + 8]     ; incarcam valoarea argumentului din stackframe

    cmp eax, 2             ; testam coditia de terminare (n < 2)
    jae .recursiv
    mov eax, 1             ; returnam 1 cand n < 2
    jmp .gata

.recursiv:
    push eax               ; retinem valoarea lui n sa nu-l pierdem la apelul recursiv

    dec eax
    push eax               ; apelam factorial(n-1), am urcat in stiva valoarea n-1
    call factorial          ; apel recursiv (STDCALL)

    mov [ebp - 4], eax      ; salvam in DWORD-ul temporar valoarea lui factorial(n-1)
    pop eax                ; restauram valoarea lui n
    mov ebx, [ebp - 4]      ; ebx <- factorial(n-1), preluat din variabila temporara
    mul ebx                ; calculam (edx:)eax = (EAX=)n * (EBX=)factorial(n-1)

.gata:
    pop ebx                ; refacem EBX la valoarea initiala
    add esp, 4             ; eliberam memoria ocupata de variabila temporara
    pop ebp                ; restauram ebp la valoarea initiala
    ret 4                  ; STDCALL: revenire cu eliberare memorie parametri
```

Interfacing with high-level languages

- Subroutine call – exit code
 - Example: exit code from a STDCALL asm function (from recursive call) -stackframe

```
factorial:
    push ebp
    mov ebp, esp           ; definim un cadru de stiva cu EBP pivot/referinta
    sub esp, 4             ; alocam spatiu pe stiva pentru o variabila DWORD temporara
    push ebx               ; salvam ebx pentru a-l putea restaura

    mov eax, [ebp + 8]     ; incarcam valoarea argumentului din stackframe

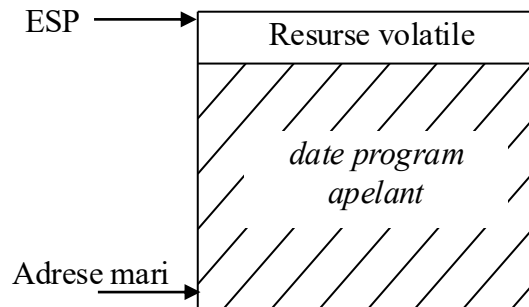
    cmp eax, 2             ; testam coditia de terminare (n < 2)
    jae .recursiv
    mov eax, 1             ; returnam 1 cand n < 2
    jmp .gata
```

```
.recursiv:
    push eax               ; retinem valoarea lui n sa nu-l pierdem la apelul recursiv

    dec eax
    push eax               ; apelam factorial(n-1), am urcat in stiva valoarea n-1
    call factorial         ; apel recursiv (STDCALL)

    mov [ebp - 4], eax     ; salvam in DWORD-ul temporar valoarea lui factorial(n-1)
    pop eax               ; restauram valoarea lui n
    mov ebx, [ebp - 4]     ; ebx <- factorial(n-1), preluat din variabila temporara
    mul ebx               ; calculam (edx:)eax = (EAX=)n * (EBX=)factorial(n-1)
```

```
.gata:
    pop ebx               ; refacem EBX la valoarea initiala
    add esp, 4            ; eliberam memoria ocupata de variabila temporara
    pop ebp               ; restauram ebp la valoarea initiala
    ret 4                 ; STDCALL: revenire cu eliberare memorie parametri
```



References

- Slides adapted from the course of Vancea Alexandru and Marius Vanța